The producer/consumer problem (2/4)

CONDIZIONI SEMPRE VERIFICATE

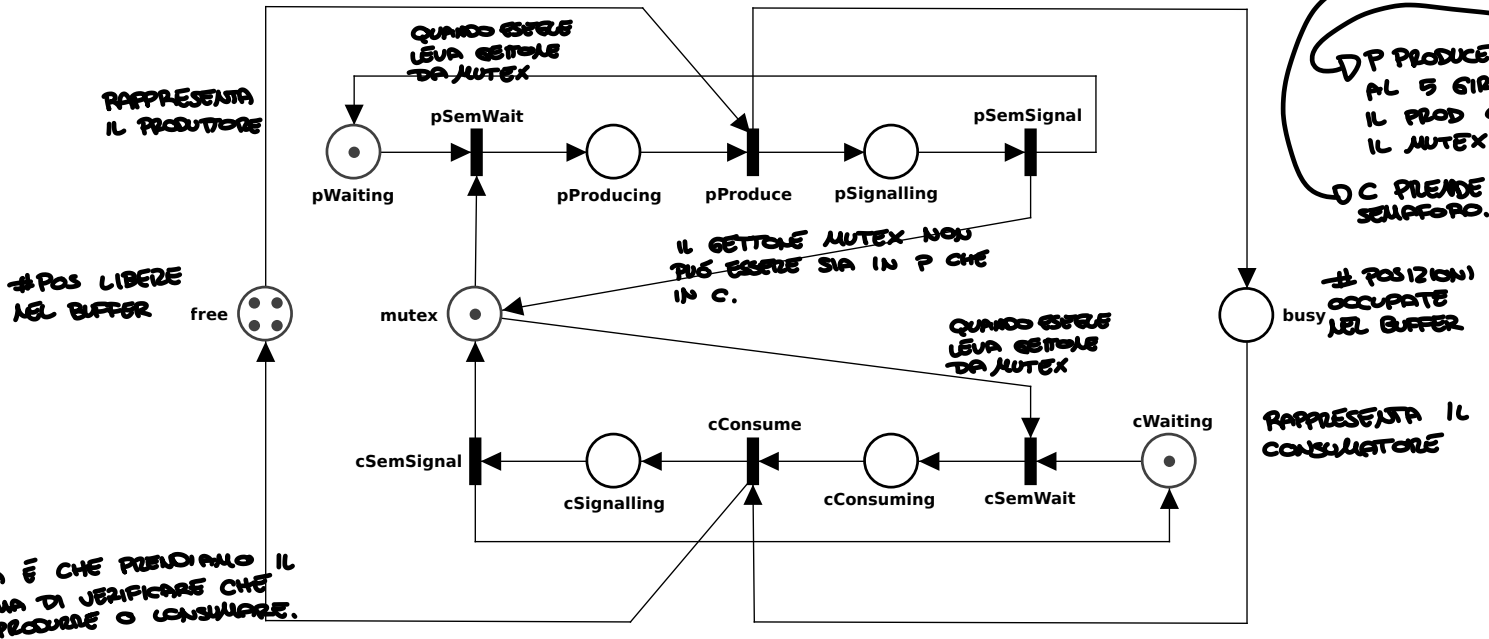
- Place invariants
  - $m(\texttt{pWaiting}) + m(\texttt{pProducing}) + m(\texttt{pSignalling}) = 1 \; \forall\, m \in \mathcal{M}$
  - $m(\texttt{cWaiting}) + m(\texttt{cProducing}) + m(\texttt{cSignalling}) = 1 \; \forall\, m \in \mathcal{M}$
  - $m(\texttt{free}) + m(\texttt{busy}) = 4 \; \forall\, m \in \mathcal{M}$ (4 is the **buffer capacity**)

C'È UN UNICO GETTONE CHE GIRA NEL PRODUTTORE E UNO NEL CONSUMATORE

- The model guarantees mutual exclusion (safety requirement satisfied)
- The model is prone to **deadlock** (liveness requirement not satisfied)
  - No enabled transition in any marking $m \mid m(\texttt{pProducing}) = 1 \wedge m(\texttt{busy}) = 4$
  - No enabled transition in any marking $m \mid m(\texttt{cConsuming}) = 1 \wedge m(\texttt{free}) = 4$

P PRODUCE 4 ITEM E AL 5 GIRO È ANCORA IL PROD CHE PRENDE IL MUTEX.

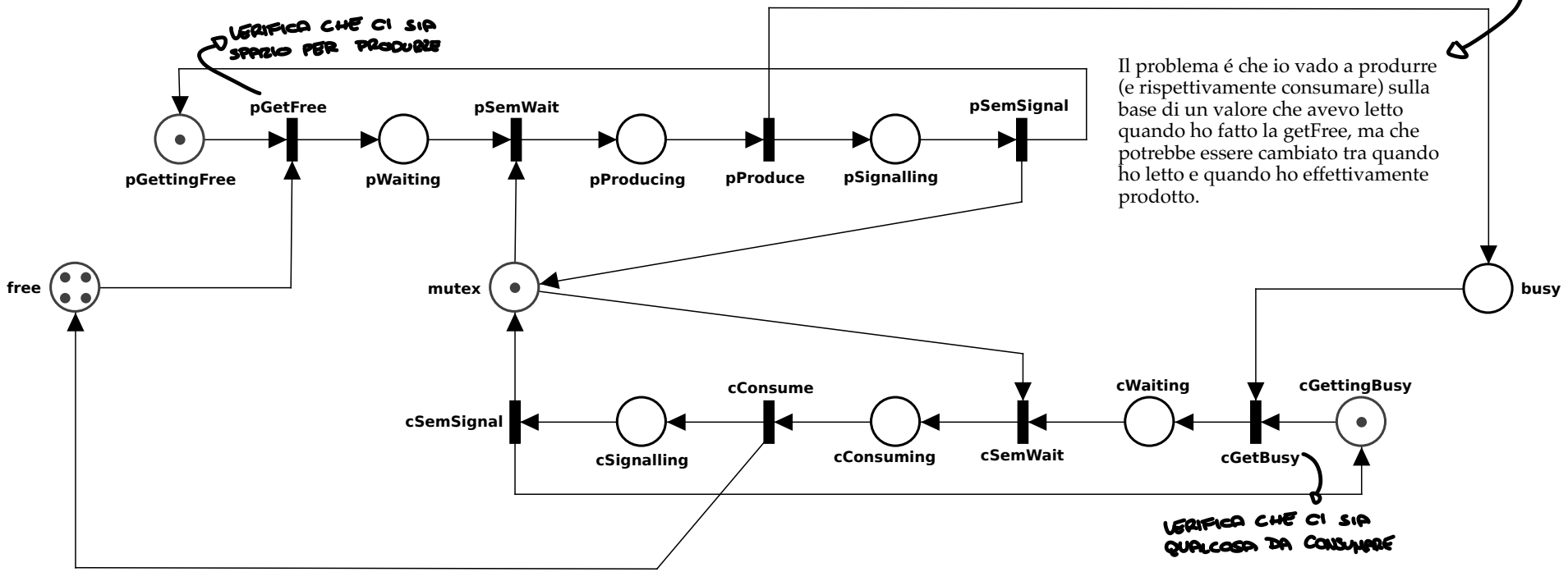
C PRENDE SUBITO IL SEMAFORO.



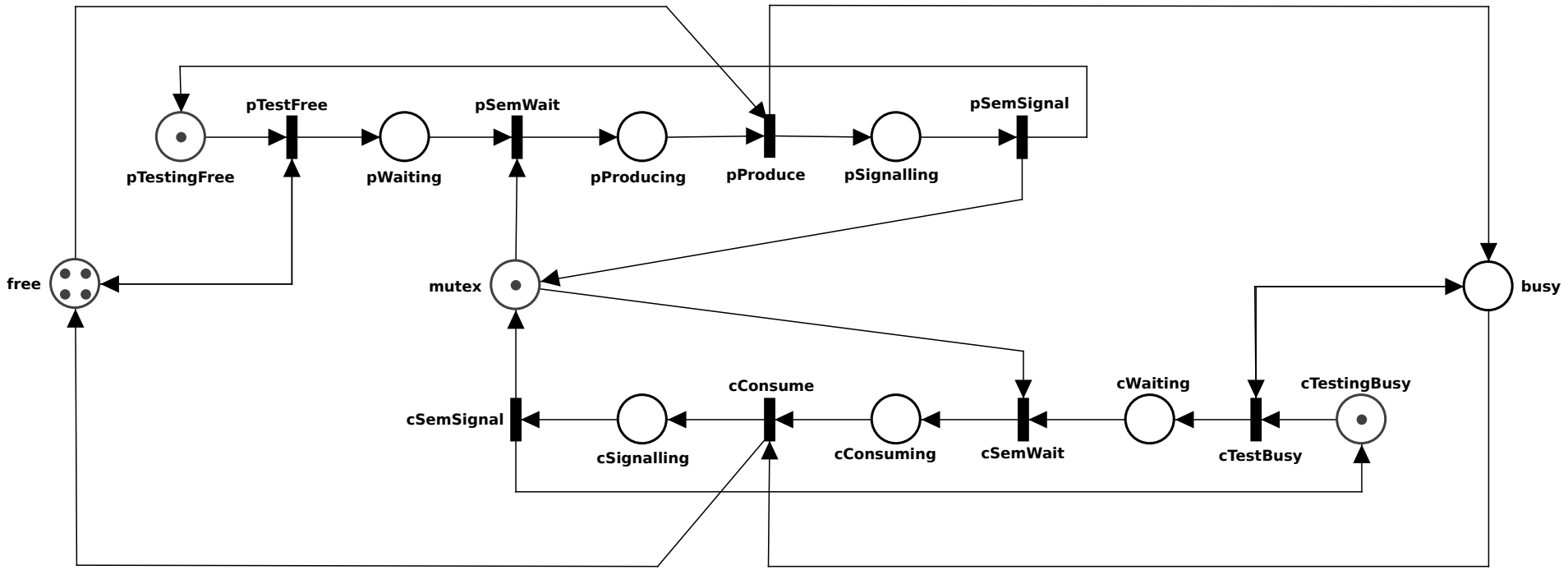
IL PROBLEMA È CHE PRENDIAMO IL MUTEX PRIMA DI VERIFICARE CHE SI POSSA PRODURRE O CONSUMARE.

The producer/consumer problem (3/4)

- The model guarantees mutual exclusion (safety requirement satisfied)
- The model prevents deadlock (liveness requirement satisfied)
- The model cannot be translated into real-time code
  - The number of current products could be represented by a shared variable `sv`
  - There may be a context switch (due to the fact that the semaphore is busy) after the producer/consumer has read the variable and before it modifies it



The producer/consumer problem (4/4)

- The model guarantees mutual exclusion (safety requirement satisfied)
- The model prevents deadlock (liveness requirement satisfied)
- The model can be translated into real-time code
  - The shared variable is first tested by the producer/consumer and then modified

É lo stesso di prima ma testFree adesso é solo un controllo e non altera il numero di gettoni in free